CURSO ITF DE TESTING MANUAL

Base de Datos Cinema

Diseño de una base de datos y consultas SQL

Resumen del proyecto

Elemento	Detalles
Estudiante	Valentina Hatos
Rol	QA Tester
Aplicación	Base de datos MySQL “cinema”
Módulo probado	Tablas, relaciones y consultas SQL
Herramientas	MySQL Workbench
Fecha	Abril de 2022

Índice

1. Introducción....3

2. Modelo de la base de datos....3

3. Creación de las tablas....3

3.1 La tabla movie....3

3.2 La tabla lead_actor....3

3.3 La tabla movie_genres....4

3.4 La tabla screening....4

4. Modificación de la estructura....4

5. Consultas....4

6. Conclusión....5

1. Introducción

Este proyecto es un ejercicio de SQL del curso ITF de Testing Manual. Creé una base de datos para un cine. La base de datos guarda películas, actores principales, géneros de películas y proyecciones.

Creé las tablas, inserté datos y modifiqué la estructura de algunas tablas. Después escribí 9 consultas para obtener información de la base de datos.

2. Modelo de la base de datos

Creé una base de datos llamada “cinema” con cuatro tablas. También creé el modelo de la base de datos en MySQL Workbench.

Tabla	Qué guarda
movie	El título, el año y la duración de cada película
lead_actor	El nombre y el apellido del actor principal de cada película
movie_genres	El género de cada película
screening	El día, la hora de inicio y la película de cada proyección

La tabla movie es la tabla principal. Las otras tres tablas están vinculadas a ella mediante claves foráneas que hacen referencia a la columna movie_id.

3. Creación de las tablas

3.1 La tabla movie

La columna movie_id es la clave primaria y aumenta automáticamente. El título de la película es obligatorio. La columna movie_time guarda la duración en minutos.

```
create database cinema;
use cinema;

create table movie (
movie_id int auto_increment primary key,
movie_title varchar(50) not null,
movie_year int,
movie_time int
);
```

Inserté 15 películas, por ejemplo Forrest Gump, Titanic, The Matrix y Avatar.

```
insert into movie (movie_title, movie_year, movie_time)
values ('Forrest Gump', 1994, 194), ('Bohemian Rapsody', 2018, 205), ('Titanic',
1997, 190), ...
```

3.2 La tabla lead_actor

Esta tabla guarda el actor principal de cada película. La columna actor_id es la clave primaria. También es una clave foránea que hace referencia a la tabla movie.

```
create table lead_actor (
actor_id int auto_increment primary key,
first_name varchar(50),
last_name varchar(50),
movie_id int not null,
foreign key (actor_id) references movie(movie_id)
);
```

Inserté 15 registros, uno para cada película. Algunos actores aparecen en más de una película: Tom Hanks en cuatro películas y Leonardo DiCaprio en tres.

3.3 La tabla movie_genres

Esta tabla guarda el género de cada película. La columna movie_ref_id es una clave foránea que hace referencia a la tabla movie.

```
create table movie_gender (
movie_ref_id int,
gender_type varchar(50),
foreign key (movie_ref_id) references movie(movie_id)
);
```

Inserté un género para cada película. Utilicé cinco géneros: Drama, Horror, Adventure, SciFi y Romantic.

3.4 La tabla screening

Esta tabla guarda las proyecciones del cine: el día, la hora de inicio, la duración y el título de la película.

```
create table screening (
screening_day date,
start_time time,
movie_time int,
movie_title varchar(50)
);
```

Inserté 11 proyecciones, entre el 25 y el 27 de julio de 2022.

4. Modificación de la estructura

Después de crear las tablas, hice algunos cambios en su estructura.

Primero, cambié el nombre de la tabla movie_gender a movie_genres. También cambié el nombre de la columna gender_type a genres_type.

```
alter table movie_gender rename to movie_genres;
alter table movie_genres rename column gender_type to genres_type;
```

Después vinculé la tabla screening a la tabla movie. Añadí una nueva columna, screening_id. La rellené con el id de cada película, mediante sentencias UPDATE. Al final, añadí la clave foránea.

```
alter table screening add screening_id int not null;

update screening set screening_id = 2 where movie_title = 'Bohemian Rapsody';
...

alter table screening add foreign key (screening_id) references movie(movie_id);
```

Después de cada cambio, verifiqué las tablas con sentencias SELECT * y DESC.

5. Consultas

Escribí 9 consultas. A continuación se muestran cada tarea, su consulta y una breve explicación.

Tarea 1: Encontrar todas las películas posteriores al año 2000 que duran más de 3 horas.

```
select * from movie where movie_year > '2000' and movie_time > '180';
```

Combiné dos condiciones con AND. La consulta devuelve dos películas: Bohemian Rapsody y Catch Me If You Can.

Tarea 2: Encontrar el nombre de la película con la mayor duración.

```
select movie_title from movie
where movie_time = (select max(movie_time) from movie);
```

Utilicé una subconsulta con MAX. Dos películas tienen la misma duración máxima, así que la consulta devuelve las dos: Bohemian Rapsody y Sleepless In Seattle.

Tarea 3: Encontrar la película más antigua.

```
select movie_title from movie order by movie_year asc limit 1;
```

Ordené las películas por año y me quedé solo con la primera fila mediante LIMIT 1. El resultado es Psycho (1960).

Tarea 4: Encontrar todas las películas, con su año, en las que actuó Tom Hanks.

```
select movie_title, movie_year from movie
left join lead_actor on movie.movie_id = lead_actor.actor_id
where lead_actor.first_name = 'Tom' and lead_actor.last_name = 'Hanks';
```

Uní las tablas movie y lead_actor y filtré por el nombre del actor. La consulta devuelve cuatro películas: Forrest Gump, Appolo 13, Sleepless In Seattle y The Terminal.

Tarea 5: Obtener el género de las películas en las que actuó Leonardo DiCaprio.

```
select genres_type
from movie_genres left join lead_actor on movie_genres.movie_ref_id =
lead_actor.actor_id
where lead_actor.first_name = 'Leonardo' and lead_actor.last_name = 'DiCaprio';
```

Uní las tablas movie_genres y lead_actor y filtré por el nombre del actor. El resultado muestra los géneros Drama y Adventure.

Tarea 6: Mostrar todos los géneros de películas distintos.

```
select * from movie_genres
group by genres_type;
```

Agrupé las filas por género, así que cada género aparece una sola vez.

Tarea 7: Obtener todas las películas que tienen una proyección en el cine.

```
select movie_title from screening;
```

Seleccioné los títulos de las películas de la tabla screening.

Tarea 8: Obtener los nombres de todas las películas que se proyectan el 26 y el 27 de julio.

```
select movie_title from screening
where screening_day = '2022-07-26' or screening_day = '2022-07-27';
```

Combiné dos fechas con OR.

Tarea 9: Calcular cuántas películas se proyectan el 25 de julio.

```
select count(*) from screening
where screening_day = '2022-07-25';
```

Utilicé la función COUNT. El resultado es 4.

El script SQL completo se puede consultar en el repositorio del proyecto (cinema database.sql). El modelo de la base de datos está disponible en cinema_model.mwb.

6. Conclusión

En este ejercicio, diseñé una base de datos con cuatro tablas relacionadas. Creé las tablas, inserté datos y modifiqué su estructura con sentencias ALTER TABLE y UPDATE. Después escribí 9 consultas con filtros, ordenación, joins, subconsultas, agrupación y funciones.

Este ejercicio me ayudó a practicar las sentencias principales de SQL. También me ayudó a entender cómo se vinculan las tablas mediante claves primarias y foráneas.